18th April : JWT Keys and OpenID Connect

-Jaani

1

Symmetric Authentication:

In this type of authentication, one secret key is used to verify user.

eg: Sender uses the same key to encrypt and the reciever uses the same key to verify/decrypt.

Asymmetric Authentication: (Public - Private Key)

Private key can generate a token, but as public key can only verify the token (belong to its private key) but does not expose private key.

Both public and private keys are different.

⇒ ~~Publ~~ Private key signs

⇒ Public key verifies.

How to distribute public key:

1. Simple way (basic API)

```
app.get("/public-key", (req, res) => {
    res.json({
        key: process.env.PUBLIC_KEY
    });
});
```

Because any one can expose/replace their ~~way~~ key, so this is lowtrust. So we need a standard.

2. OIDC (Open ID Connect)

OIDC is built on OAuth 2.0 and uses JSON-based standards

⇒2.1. service discovery : GET /.well-known/openid-configuration

Return JSON { authorization_url : '')

issuer : base url , eg https://chaiauth.com .

authorization_endpoint : Host your login page , eg https://chaiauth.com/login

~~token~~ userinfo_endpoint : If you want the token of user, use this

jwks_uri : Here you will get the public key

OIDC gives us standardization : token will have :

"aud"

"email"

"sub" : user url etc etc.

If we have a express application.

we need to make a

GET .well-known/open-id-configuration

Return JSON Response will have {

issuer : Who is basically, who will issue the token. To verify its coming from trusted place

authorization_endpoint : A endpoint where user can get token assigned and authorized.

~~token~~ userinfo_endpoint : If you want to get user info, if you have the token.

jwks_uri : This is where my public keys are available

Steps: Client redirects the user to the authorization server for auth.

1. https://chaiauth.com/.well-known/openid-configuration

Service Discovery Endpoints

Redirect user to { https://chaiauth.com/login }

But problem with this, anyone can access/gets verified if they have token, so we solved these problems with OAuth 2.0.

By default, OIDC is secure, if login page is not exposed.

SSO (Single Sign-On)

You login once, and then you can access multiple services without logging in again and again.

OAuth 2.0

If me as chaiauth.com, wants to use OIDC, i need to login into developer console and fill details: application name, url, privacy, terms, user data, where is your application hosted etc. I can get a →

⇒ client-id

⇒ client-secret.

Now, client-id is sent in url, when someone does, "login with google" to verify the request belongs to.

1. Not secure way to give ~~back to~~ back token ⇒

xyz.com/api/redirect-uri/?token = ...

can expose attacks.

⇒ OAuth 2.0 never gives you token, it gives you a short code valid for next 1 minute. User can ~~send~~ now send that code. So now code + client-secret is sent to google if verified, a token is exchanged at client's backend.

The short lived code is sent as redirect-uri ?code=.....
Now the client's backend calls token-endpoint and 4
token exchange takes place, and a access token is given in exchange.

Assignment : Build your own Clerk / Auth0 / Better Auth

#SAML (Security Assertion Markup Language)

This is XML-based, and is mostly used in enterprises

#Two important endpoints.

1. Authorization-endpoint : The user login happens here
2. Token-endpoint : server to server token exchange

In simple terms, OAuth 2.0 is an authorization framework that allows applications to access resources without sharing user credentials.

Here OpenID helps to provide authentication by issuing ID tokens.

OIDC enables SSO across multiple applications.

Follow: jaaaanitot